

2^e année Licence Informatique Décisionnelle en Santé Digitale

Module : Sciences de Données Médicales par Python

Rapport du Projet

Prism Medical Data Lab

Application web interactive pour l'analyse, le nettoyage et le preprocessing des données médicales

Technologies : Python, Streamlit, Pandas, NumPy, Plotly, Matplotlib,
OpenPyXL

Année universitaire 2025–2026

Table des matières

1	Introduction	1
2	Problématique	1
3	Objectifs du projet	1
4	Technologies utilisées	1
4.1	Python	2
4.2	Streamlit	2
4.3	Pandas et NumPy	2
4.4	Plotly et Matplotlib	2
4.5	OpenPyXL	2
4.6	Pytest	2
5	Architecture du projet	2
5.1	Rôle des fichiers principaux	3
6	Description des fonctionnalités	3
6.1	Page Accueil	3
6.2	Importation des données	3
6.3	Analyse exploratoire	4
6.4	Preprocessing	4
6.5	Visualisations	4
6.6	Exportation	4
7	Dataset d'exemple	4
8	Méthodes de preprocessing	4
8.1	Valeurs manquantes	4
8.2	Doublons	5
8.3	Outliers	5
8.4	Normalisation et standardisation	5
9	Tests réalisés	5

10 Installation et lancement	6
11 Scénario de démonstration vidéo	6
12 Points forts	6
13 Limites et améliorations possibles	6
14 Conclusion	7

1 Introduction

Les établissements de santé produisent de grandes quantités de données : dossiers patients, résultats biologiques, données de suivi clinique, indicateurs hospitaliers et informations de santé publique. Avant toute exploitation décisionnelle, ces données doivent être explorées, nettoyées, transformées et visualisées.

Ce projet propose une application web interactive appelée **Prism Medical Data Lab**. Elle permet à l'utilisateur d'importer un fichier CSV médical, de comprendre rapidement sa structure, d'identifier les problèmes de qualité, d'appliquer des opérations de preprocessing et d'exporter les résultats.

2 Problématique

Les datasets médicaux contiennent souvent des valeurs manquantes, des doublons, des colonnes mal typées, des variables catégorielles et des valeurs aberrantes. Ces problèmes peuvent fausser les statistiques et réduire la fiabilité des analyses.

La problématique du projet est donc la suivante :

Comment développer une application simple, interactive et complète permettant d'analyser, nettoyer, prétraiter, visualiser et exporter des données médicales tabulaires avec Python ?

3 Objectifs du projet

L'objectif général est de créer une plateforme interactive dédiée aux sciences de données médicales. Les objectifs spécifiques sont :

- importer des fichiers CSV ;
- afficher un aperçu complet du dataset ;
- détecter automatiquement les colonnes numériques et catégorielles ;
- calculer les statistiques descriptives ;
- analyser les corrélations entre variables numériques ;
- détecter les valeurs manquantes et les doublons ;
- appliquer des techniques de nettoyage et de preprocessing ;
- générer des visualisations interactives ;
- exporter les données nettoyées et les rapports d'analyse ;
- proposer une interface claire, moderne et organisée.

4 Technologies utilisées

4.1 Python

Python est utilisé comme langage principal. Il permet de développer rapidement des solutions de data science grâce à son écosystème riche.

4.2 Streamlit

Streamlit sert à créer l'interface web interactive : sidebar, pages, boutons, tableaux, graphiques, messages et boutons de téléchargement.

4.3 Pandas et NumPy

Pandas est utilisé pour lire, manipuler et exporter les données. NumPy est utilisé pour certaines opérations numériques comme la standardisation.

4.4 Plotly et Matplotlib

Plotly permet de générer des graphiques interactifs : histogrammes, boxplots, scatter plots, pie charts et heatmaps. Matplotlib est prévu comme bibliothèque complémentaire de visualisation.

4.5 OpenPyXL

OpenPyXL permet à Pandas de générer des exports Excel.

4.6 Pytest

Pytest est utilisé pour tester automatiquement les fonctions principales de l'application.

5 Architecture du projet

Le projet est organisé selon une architecture modulaire :

```
project/
|-- app.py
|-- pages/
|-- utils/
| |-- data_loader.py
| |-- preprocessing.py
| |-- statistics.py
| |-- visualization.py
| |-- exporter.py
| '-- errors.py
```

```
|-- data/
| '-- sample_medical_data.csv
|-- exports/
|-- assets/
|-- tests/
| |-- conftest.py
| '-- test_processing.py
|-- requirements.txt
|-- README.md
|-- RUN_RECORD.md
'-- COMPTE_RENDU.md
```

5.1 Rôle des fichiers principaux

Fichier	Rôle
app.py	Interface Streamlit principale et navigation entre les pages.
utils/data_loader.py	Chargement CSV et résumé global du dataset.
utils/statistics.py	Statistiques descriptives, corrélations et valeurs manquantes.
utils/preprocessing.py	Nettoyage, transformation, encodage et gestion des outliers.
utils/visualization.py	Création des graphiques interactifs.
utils/exporter.py	Export CSV, Excel et rapport texte.
tests/test_processing.py	Tests unitaires des fonctions critiques.

6 Description des fonctionnalités

6.1 Page Accueil

La page d'accueil présente l'application et affiche des indicateurs clés : nombre de lignes, nombre de colonnes, doublons et mémoire utilisée. Elle donne aussi un aperçu initial des données.

6.2 Importation des données

L'utilisateur peut importer un fichier CSV. Après importation, l'application affiche les dimensions, les types des colonnes, les colonnes numériques, les colonnes catégorielles, les doublons et la mémoire utilisée.

6.3 Analyse exploratoire

Cette partie permet de comprendre le dataset à travers les statistiques descriptives, la matrice de corrélation, la heatmap, l'analyse des valeurs manquantes et l'affichage des lignes dupliquées.

6.4 Preprocessing

La page preprocessing permet de gérer les valeurs manquantes, supprimer les doublons, détecter et supprimer les outliers par méthode IQR, normaliser, standardiser, encoder les variables catégorielles, renommer des colonnes, changer les types et sélectionner les variables utiles.

6.5 Visualisations

L'application propose des histogrammes, boxplots, scatter plots, bar charts, pie charts, heatmaps de corrélation et séries temporelles. Les graphiques sont interactifs grâce à Plotly.

6.6 Exportation

Les résultats peuvent être téléchargés sous forme de dataset nettoyé en CSV, dataset nettoyé en Excel et rapport d'analyse au format texte.

7 Dataset d'exemple

Le fichier `data/sample_medical_data.csv` représente un petit dataset médical contenant les colonnes suivantes : identifiant patient, âge, sexe, IMC, glucose, tension artérielle, cholestérol, diagnostic et date de visite.

Ce dataset contient volontairement quelques valeurs manquantes et un doublon afin de tester les fonctionnalités de nettoyage.

8 Méthodes de preprocessing

8.1 Valeurs manquantes

L'application propose plusieurs stratégies : suppression des lignes, suppression des colonnes, remplacement par moyenne, remplacement par médiane, remplacement par mode ou remplacement par une valeur personnalisée.

8.2 Doublons

Les lignes dupliquées sont détectées avec Pandas puis supprimées automatiquement lorsque l'utilisateur choisit cette option.

8.3 Outliers

Les outliers sont détectés avec la méthode IQR. Pour une variable numérique, on calcule le premier quartile Q1, le troisième quartile Q3 et l'écart interquartile IQR. Les valeurs inférieures à $Q1 - 1.5 \text{ IQR}$ ou supérieures à $Q3 + 1.5 \text{ IQR}$ sont considérées comme aberrantes.

8.4 Normalisation et standardisation

La normalisation Min-Max ramène les valeurs entre 0 et 1. La standardisation Z-score centre les valeurs autour de 0 avec un écart-type proche de 1.

9 Tests réalisés

Les tests unitaires vérifient les fonctions principales :

- détection des dimensions et des types de colonnes ;
- détection des doublons ;
- calcul des valeurs manquantes ;
- statistiques descriptives ;
- matrice de corrélation ;
- traitement des valeurs manquantes ;
- encodage catégoriel ;
- suppression des doublons et des outliers ;
- normalisation et standardisation ;
- export CSV, Excel et rapport.

Commande de validation :

```
pytest -q
```

Résultat obtenu dans l'environnement de travail :

```
4 passed, 1 skipped
```

Le test Excel est ignoré localement si `openpyxl` n'est pas installé. Après installation des dépendances, l'export Excel est disponible.

10 Installation et lancement

Installation des dépendances :

```
pip install -r requirements.txt
```

Lancement de l'application :

```
streamlit run app.py
```

Après lancement, Streamlit affiche une URL locale, généralement `http://localhost:8501`.

11 Scénario de démonstration vidéo

Pour réaliser une vidéo de démonstration d'environ cinq minutes :

1. Présenter la page d'accueil et l'objectif de l'application.
2. Montrer les KPI cards : lignes, colonnes, doublons et mémoire.
3. Importer un fichier CSV médical.
4. Afficher l'aperçu du dataset et les types des colonnes.
5. Présenter les statistiques descriptives et la heatmap.
6. Montrer les valeurs manquantes et les doublons.
7. Appliquer un traitement des valeurs manquantes.
8. Supprimer les doublons puis appliquer une normalisation ou un encodage.
9. Générer un histogramme, un boxplot et une heatmap.
10. Exporter le dataset nettoyé et le rapport d'analyse.

12 Points forts

- Interface simple organisée avec une sidebar.
- Architecture claire et modulaire.
- Application utilisable avec tout fichier CSV tabulaire.
- Dataset médical d'exemple intégré.
- Fonctions de nettoyage adaptées aux problèmes fréquents des données médicales.
- Graphiques interactifs faciles à interpréter.
- Exportation des résultats pour réutilisation externe.
- Tests unitaires pour valider les fonctions critiques.

13 Limites et améliorations possibles

Les améliorations possibles sont :

- ajouter l'importation de fichiers Excel ;
- exporter les graphiques directement en PNG ;
- générer un rapport PDF automatiquement ;
- ajouter une analyse spécifique pour les images biomédicales ;
- intégrer des modèles de machine learning ;
- ajouter des filtres dynamiques plus avancés.

14 Conclusion

Le projet **Prism Medical Data Lab** répond aux exigences d'une application de sciences de données médicales développée avec Python. Il permet d'importer, explorer, nettoyer, transformer, visualiser et exporter des données médicales tabulaires à travers une interface Streamlit claire et interactive.

Grâce à son architecture modulaire, le projet est facile à maintenir et à améliorer. Les tests ajoutés assurent la fiabilité des fonctions principales. L'application constitue une base solide pour l'analyse exploratoire et le preprocessing des données médicales dans un contexte académique ou professionnel.